Logistics & Rider Module Task Assignment Guide

Everything unbuilt in the project plan lives in two modules — Logistics/Sorting Center and Rider/Courier. This guide breaks both into features, tasks, owners, estimates, and full task details ready to raise straight into the project board.

Module 1 — Logistics / Sorting Center

*Owner: **Amaru Jay Balmes** (frontend & coordination) · backend scaffolding: **Jay Mark Del Valle***

1.1 Database & data layer

No dependencies — do this first.

Create sorting_centers migration & model

Jay Mark · 2h

Create the sorting_centers migration and Eloquent model. Fields: hub name, address (region/province/city/barangay matching PSGC dropdowns), DTI permit file path, coverage barangay list, status (PENDING/APPROVED/REJECTED). Relationships: hasMany parcels, hasMany rider_area_assignments. First task in the module — everything else depends on it.

Done when: migration runs clean, model relationships resolve, a hub record can be created/queried via tinker.

Create parcels migration & model

Jay Mark · 2h

Create the parcels migration and model. Fields: tracking number, order reference, current status (AT_SORTING_CENTER / SORTED / ASSIGNED_TO_RIDER / OUT_FOR_DELIVERY / DELIVERED / DELIVERY_FAILED / RTS), destination barangay, sorting_center_id, assigned_rider_id, scanned_in_at, pickup_rider_id. Relationships: belongsTo sorting_center, belongsTo rider.

Done when: migration runs clean and a parcel record links correctly to a hub and a rider.

Create rider_area_assignments migration & model

Jay Mark · 2h

Create the rider_area_assignments migration and model. Fields: rider_id, sorting_center_id, coverage area/barangay list, active flag. Relationship: belongsTo rider, belongsTo sorting_center.

Done when: a rider can be linked to one or more coverage areas under a specific hub.

Define relationships (hub → parcels, rider → area)

Jay Mark · 1h

Wire up and test all Eloquent relationships across sorting_centers, parcels, riders, and rider_area_assignments — hasMany, belongsTo, and any pivot tables needed. Confirm eager loading works (with()) for the dashboards built in later tasks.

Done when: $hub->parcels, $rider->areas, and $parcel->rider all return correct data in tinker.

1.2 Sorting center registration flow

Depends on 1.1

Feature done when: A rep submits hub info + permit + coverage areas, sees the “wait for admin approval” message, and the record shows up in Admin's queue as PENDING.

Registration controller + validation rules

Jay Mark · 3h

Build the controller and form request for hub registration. Validate: required address fields, valid PSGC codes, permit file type/size, at least one coverage barangay selected. Return field-level errors the frontend form can display inline.

Done when: submitting invalid data returns clear per-field errors; valid data creates a PENDING hub record.

Build hub registration form: name, address, DTI permit upload, coverage barangays

Amaru · 5h

Build the registration page/form with fields for hub name, cascading address dropdowns, DTI permit file upload (with preview), and a multi-select for coverage barangays. Match the visual style of existing registration forms in the app.

Done when: a rep can fill every field, upload a permit, and submit without a page reload error.

Wire PSGC cascading address dropdowns into the form

Amaru · 2h

Connect region → province → city → barangay dropdowns to the PSGC dataset already used elsewhere in the app, so each level filters based on the one above it.

Done when: selecting a region narrows province options correctly, all the way down to barangay.

Approval-gate notice + pending screen copy

Amaru · 1h

After submission, show a clear “your application is pending Admin approval” screen with the submitted summary and no way to access hub features yet.

Done when: a newly registered hub sees the pending screen and cannot reach /hub/dashboard.

1.3 Admin hub approval queue

Depends on 1.2

Feature done when: Admin approves or rejects with a reason, and the hub account only activates after approval.

Build AdminSortingCenterApplicationsPage (list, inspect permit, approve/reject)

Amaru · 4h

Build the admin-facing queue listing all PENDING hub applications, with a detail view to inspect the submitted permit and address info, plus approve/reject actions.

Done when: Admin can open any pending application, view the permit file, and approve or reject it from the same screen.

Approve/reject endpoint + applicant email notification

Jay Mark · 2h

Build the endpoint that flips a hub's status to APPROVED or REJECTED (with a required reason on reject), and trigger an email notification to the applicant either way.

Done when: approving activates the hub account; rejecting sends the reason and blocks login.

1.4 Hub command dashboard

Depends on 1.3

Feature done when: Every card reflects real counts, not placeholders.

Build /hub/dashboard layout & nav

Amaru · 3h

Build the base dashboard layout and navigation for approved hubs — sidebar/nav linking to intake, sorting, dispatch, riders, and reports sections built in later tasks.

Done when: an approved hub can log in and land on a working dashboard shell with working nav links (even to empty pages).

Metrics endpoint: inbound, in-sorting, active couriers

Jay Mark · 2h

Build an endpoint returning live counts: parcels inbound today, parcels currently in sorting, and active riders under this hub.

Done when: the endpoint returns accurate numbers matching a manual count in the database.

Wire dashboard cards to live data

Amaru · 2h

Connect the dashboard's summary cards to the metrics endpoint so numbers update on load/refresh instead of showing placeholder values.

Done when: every card on the dashboard reflects real, current data.

1.5 Rider vetting & area mapping (hub side)

Needs rider registration — feature 2.2

Feature done when: A hub manager approves a pending rider, assigns barangay coverage, and the rider flips to ACTIVE.

Build /hub/riders vetting queue (approve/disapprove applicants)

Amaru · 4h

Build the hub-side page listing PENDING rider applications for this hub, with document review and approve/disapprove actions.

Done when: a hub manager can review a rider's submitted documents and approve or disapprove from this screen.

Build rider-to-area assignment UI (Area A/B/C)

Amaru · 3h

Build the UI for assigning an approved rider to one or more coverage areas (e.g. Area A/B/C or specific barangays) under this hub.

Done when: assigning an area to a rider is reflected immediately in that rider's profile.

Endpoints: approve rider, assign coverage area

Jay Mark · 3h

Build the backend endpoints for rider approval/disapproval and for saving area assignments, tying into the rider_area_assignments table from 1.1.

Done when: approving a rider flips their status to ACTIVE and area assignment persists correctly.

1.6 Parcel intake & inward scanning

Depends on 1.1

Feature done when: Scanning a valid tracking number updates the order and stamps scanned_in_at.

Barcode / tracking-number scan input

Amaru · 3h

Build a scan-style input (camera or manual entry) on the hub intake screen that looks up a parcel by tracking number.

Done when: entering or scanning a valid tracking number pulls up the matching parcel record.

Endpoint: mark parcel AT_SORTING_CENTER on scan

Jay Mark · 2h

Build the endpoint that updates a parcel's status to AT_SORTING_CENTER and stamps scanned_in_at when scanned at intake.

Done when: a successful scan updates the order status visibly and records the timestamp.

1.7 Area destination sorting

Depends on 1.6

Feature done when: A dispatcher selects a batch bound for the same barangay and marks it SORTED in one action.

Sorting view grouping parcels by destination barangay

Amaru · 4h

Build a view that groups all AT_SORTING_CENTER parcels by destination barangay, so a dispatcher can see batches at a glance.

Done when: parcels are correctly grouped and the grouping updates as new parcels are scanned in.

Endpoint: bulk-update parcels to SORTED with area code

Jay Mark · 2h

Build an endpoint that accepts a batch of parcel IDs and flips them all to SORTED, tagging them with the relevant area code.

Done when: selecting a batch and confirming updates every parcel in that batch in one action.

1.8 Area-to-rider assignment

Depends on 1.5, 1.7

Feature done when: A batch is assigned only to a rider whose coverage area matches, and it appears in that rider's queue immediately.

Dispatch UI: pick sorted batch → assign to matching rider

Amaru · 4h

Build the dispatch screen where a SORTED batch can be assigned to a rider — only showing riders whose coverage area matches the batch's area code.

Done when: the rider list shown is filtered correctly by area match.

Endpoint: bulk-assign to ASSIGNED_TO_RIDER, notify rider

Jay Mark · 2h

Build the endpoint that assigns a batch to a rider, updates parcel status to ASSIGNED_TO_RIDER, and sends a notification to that rider.

Done when: the rider sees the new batch in their queue (feature 2.6) immediately after assignment.

1.9 Delivery monitoring & triage

Needs rider outcome endpoints — feature 2.7

Feature done when: Dispatcher sees live status on every out-for-delivery parcel and can trigger RTS on failed ones.

Live out-for-delivery tracker + failure log

Amaru · 4h

Build a live view of all OUT_FOR_DELIVERY parcels with status, plus a log of any that come back as DELIVERY_FAILED with the rider's reason code.

Done when: a failed delivery from feature 2.7 appears here with its reason, without a page refresh needed on the dispatcher's next load.

Return-to-seller (RTS) trigger action

Amaru + Jay Mark · 2h

Build the action that lets a dispatcher mark a failed parcel as RTS (Return to Seller), updating status and notifying the seller.

Done when: triggering RTS updates the parcel status and the seller sees the change in their own order view.

1.10 Logistics reports

Depends on 1.6–1.9

Feature done when: Report figures for a chosen range match a manual count of parcels in that range.

Report page: throughput, on-time rate, rider efficiency + date range

Amaru · 4h

Build a reporting page showing parcel throughput, on-time delivery rate, and per-rider efficiency, filterable by a custom date range.

Done when: changing the date range recalculates every figure on the page.

Reports aggregation endpoint

Jay Mark · 3h

Build the backend endpoint(s) that aggregate parcel and delivery data into the metrics the report page needs.

Done when: the endpoint returns correct totals for any given date range, verified against raw table counts.

QA: verify report totals against raw parcel data

King · 2h

Manually cross-check report figures for at least two different date ranges against a manual count/query of the raw parcels table.

Done when: all checked figures match exactly; any mismatch is filed as a bug against the aggregation endpoint.

Module 2 — Rider / Courier

*Owners: **King Ordoña** (QA & integration) and **Jef Liam Rubiales** (ops & business logic) · backend scaffolding: **Jay Mark Del Valle***

2.1 Database & data layer

No dependencies — do this first.

Rider profile table + vehicle fields migration

Jay Mark · 2h

Create the rider profile migration/model with fields: name, contact info, address, vehicle type, plate number, OR/CR file, driver's license file, status (PENDING/ACTIVE/DISAPPROVED).

Done when: migration runs clean and a rider record can be created/queried via tinker.

rider_earnings / delivery_logs migration

Jay Mark · 2h

Create migrations for rider_earnings (per-delivery fee, date, rider_id) and delivery_logs (parcel_id, rider_id, outcome, timestamp, notes).

Done when: both tables migrate cleanly and link correctly to riders and parcels.

2.2 Rider registration flow

Depends on 2.1

Feature done when: A rider submits all fields + documents, sees the logistics-approval notice, and lands in the hub's vetting queue (feature 1.5) as PENDING.

Build CourierRegisterPage: personal info, address dropdowns, vehicle type, plate no.

Liam · 5h

Build the rider registration page with personal info fields, cascading PSGC address dropdowns, vehicle type selector, and plate number field.

Done when: all fields are present, validated client-side, and submit correctly.

OR/CR + Driver's License upload component

Liam · 2h

Build a reusable file upload component for OR/CR and Driver's License documents, with preview and file-type/size validation.

Done when: both documents can be uploaded, previewed, and are attached correctly to the submission.

Validation rules + submit endpoint

Jay Mark · 3h

Build the form request validation and submission endpoint for rider registration, creating a PENDING rider record.

Done when: invalid submissions return clear field errors; valid ones create a rider record visible in the hub's vetting queue (1.5).

Approval-gate copy: “wait for the Logistics/Sorting Center's approval”

Liam · 1h

Add the post-submission screen telling the rider their application is pending approval from the Logistics/Sorting Center, with no access to rider features yet.

Done when: a newly registered rider sees this screen and can't reach /rider/dashboard.

2.3 Rider authentication

Depends on 2.2

Feature done when: An unapproved rider can't log in; an approved one reaches their dashboard.

Mobile-friendly rider login

King · 2h

Build a login screen optimized for mobile use (riders will mostly use this on phones), reusing existing auth patterns where possible.

Done when: an approved rider can log in from a mobile viewport without layout issues.

Session/token checks + role-guard middleware

Jay Mark · 1h

Add middleware that blocks unapproved riders from any rider-only route, and confirms session/token validity on each request.

Done when: a PENDING or DISAPPROVED rider is redirected away from rider routes; an ACTIVE rider passes through.

2.4 Pickup dashboard

Depends on 2.3

Feature done when: A rider only sees pickups in their own coverage area and accepting one moves it to “collected.”

“Items for Pickup” console: seller locations, time slots, parcel counts

Liam · 4h

Build the rider's pickup console showing pending pickup requests with seller location, requested time slot, and parcel count per request.

Done when: the console lists all pending pickups assigned/available to this rider.

Accept / decline pickup request action

Liam · 2h

Add accept/decline buttons on each pickup request; accepting should claim it for this rider, declining should return it to the pool.

Done when: accepting moves the request into this rider's active pickups; declining removes it from their view.

Endpoint: list pickup requests for this rider

Jay Mark · 2h

Build the endpoint returning only pickup requests within this rider's coverage area (per rider_area_assignments).

Done when: the endpoint never returns requests outside the rider's assigned area.

QA: pickup list only shows requests in the rider's coverage area

King · 2h

Test with riders assigned to different areas and confirm each only sees their own area's pickup requests.

Done when: cross-area leakage is confirmed absent across at least two test riders.

2.5 Pickup execution flow

Depends on 2.4

Feature done when: Confirming handover flips the order to PICKED_UP and the seller sees it update live.

Scan/verify parcel at seller, confirm-handover UI

Liam · 3h

Build the in-app flow for verifying a parcel at pickup (scan or manual entry) and confirming handover from the seller.

Done when: a rider can complete the full pickup confirmation flow for a single parcel without errors.

Endpoint: mark PICKED_UP, log pickup_rider_id

Jay Mark · 2h

Build the endpoint that updates parcel status to PICKED_UP and records which rider picked it up and when.

Done when: the seller's order view reflects PICKED_UP status immediately after confirmation.

QA: end-to-end test seller-handover → hub-arrival chain

King · 2h

Run a full manual test: rider accepts pickup → confirms handover → parcel arrives and is scanned in at the hub (1.6).

Done when: the chain completes with correct status at every step and no orphaned records.

2.6 Delivery dashboard

Depends on 1.8

Feature done when: The rider's queue exactly matches what the hub assigned them, in a sensible order.

“Items for Delivery” console: assigned parcels, recipient info, dispatch order

Liam · 4h

Build the rider's delivery console listing parcels assigned to them (from 1.8), with recipient name/address and a sensible dispatch order.

Done when: a rider sees exactly the parcels the hub assigned them, in the order the hub intended.

QA: queue matches the hub's area-to-rider assignments

King · 2h

Confirm that what appears in a rider's delivery queue exactly matches what was assigned via feature 1.8, with no missing or extra parcels.

Done when: at least two assignment batches are verified end-to-end.

2.7 Delivery outcome branching

Depends on 2.6

Feature done when: A failed delivery requires a reason before submitting, and immediately shows up in feature 1.9's triage view.

Success flow: mark DELIVERED, COD collection, photo/signature proof

Liam · 4h

Build the successful-delivery flow: mark DELIVERED, capture COD amount if applicable, and capture photo or signature proof of delivery.

Done when: a completed delivery has proof attached and the order status updates to DELIVERED.

Failure flow: DELIVERY_FAILED + mandatory reason code selector

Liam · 3h

Build the failed-delivery flow requiring a reason code (e.g. recipient unavailable, wrong address, refused) before the failure can be submitted.

Done when: submission is blocked until a reason is selected, and the failure reaches the hub's triage view (1.9).

Endpoints for both outcomes + reschedule/RTS trigger

Jay Mark · 3h

Build the backend endpoints for both DELIVERED and DELIVERY_FAILED outcomes, including any reschedule or RTS trigger logic.

Done when: both flows persist correctly and notify the relevant parties (buyer, hub, seller as applicable).

QA: test every failure-reason path, confirm it hits the hub triage view

King · 3h

Test each individual failure reason code and confirm each one surfaces correctly in the Logistics triage view (1.9).

Done when: every reason code has been tested at least once end-to-end.

2.8 Buyer confirmation loop

Depends on 2.7

Feature done when: The button only appears after DELIVERED, and confirming starts the 7-day escrow countdown.

Buyer “Confirm Order Received” button (DELIVERED → COMPLETED)

King · 2h

Add a “Confirm Order Received” button on the buyer's order view, visible only after status is DELIVERED, that flips status to COMPLETED.

Done when: the button is hidden until DELIVERED and works correctly once shown.

Endpoint + escrow-release trigger tie-in

Jay Mark · 2h

Build the confirmation endpoint and tie it into the existing escrow logic to start the release countdown.

Done when: confirming triggers the 7-day escrow countdown correctly.

2.9 Earnings & delivery history

Depends on 2.7

Feature done when: Earnings for a period equal completed deliveries × the applicable fee.

Earnings dashboard: daily/weekly/monthly, completed runs

Liam · 4h

Build the rider's earnings dashboard with daily/weekly/monthly views and a breakdown of completed delivery runs.

Done when: switching views recalculates totals correctly for the selected period.

Delivery history log (past deliveries, cancellations, RTS)

King · 3h

Build a history log listing all past deliveries for this rider, including cancellations and RTS cases, with outcome and date.

Done when: the log includes every past delivery attempt, not just successful ones.

Earnings calculation endpoint

Jay Mark · 2h

Build the endpoint that calculates a rider's earnings for a given period based on completed deliveries and the applicable fee structure.

Done when: calculated totals match a manual calculation for a sample period.

QA: earnings reconcile against completed-delivery count

King · 2h

Cross-check earnings dashboard totals against a manual count of completed deliveries × fee for at least two riders.

Done when: figures reconcile exactly; discrepancies are filed as bugs.

2.10 Rider chat & account

Depends on 2.3

Feature done when: A rider can message a dispatcher from an active delivery without leaving the screen.

Wire existing chat system into rider role (dispatcher/buyer/seller threads)

King · 3h

Integrate the app's existing chat system so riders can message dispatchers, buyers, or sellers from an active delivery/pickup context.

Done when: a rider can open a chat thread tied to a specific active delivery without navigating away from the delivery screen.

Vehicle detail update + logout

Liam · 2h

Build the rider account screen for updating vehicle details (type, plate number) and a working logout action.

Done when: vehicle info updates persist correctly and logout clears the session.

Dependency Order

1. Both DB layers first (1.1 and 2.1) — nothing else can start without them.
2. Rider registration (2.2) must exist before hub vetting (1.5) has anyone to vet.
3. Hub approval (1.3) → dashboard (1.4) → intake (1.6) → sorting (1.7) → area assignment (1.8) is a straight chain inside Logistics.
4. Rider pickup (2.4–2.5) can run in parallel with Logistics 1.6–1.7, since pickup doesn't need sorting to be finished.
5. Rider delivery (2.6) needs 1.8 done — riders can't see a delivery queue until the hub has assigned parcels to them.
6. Logistics triage (1.9) needs 2.7 done — the hub can't monitor delivery outcomes that don't exist yet.
7. Reports on both sides (1.10, 2.9) come last — they aggregate data every other feature produces.

Suggested Timeline

Four weeks, running Logistics and Rider in parallel where the dependency chain above allows it.

Week	Logistics (Amaru)	Rider (King / Liam)
1	1.1 → 1.4 (DB through hub dashboard)	2.1 → 2.3 (DB through login)
2	1.5 → 1.6 (rider vetting, intake)	2.4 → 2.5 (pickup dashboard & execution)
3	1.7 → 1.8 (sorting, area assignment)	2.6 → 2.7 (delivery dashboard & outcomes)

Week	Logistics (Amaru)	Rider (King / Liam)
4	1.9 → 1.10 (triage, reports)	2.8 → 2.10 (confirmation, earnings, chat)

Required Submission

The six deliverables the milestone asks for, and where each one already lives in this guide (or how to produce it from what's here).

Deliverable	What it needs to show	Where it lives here
Project Management Board	The complete project task list	Raise every task block above as one ticket each into your project board, using the Assignee, estimate, and Done-when text already written for it.
Feature Breakdown	Modules → features → tasks → subtasks	The Module 1 / Module 2 sections: each numbered feature (1.1–1.10, 2.1–2.10) with its task blocks underneath.
Task Assignment Matrix	Who is responsible for each task	The Assignee line under every task title (Jay Mark / King / Amaru / Liam).
Project Timeline	Start/end dates, milestones, dependencies	The Suggested Timeline section (4-week table) plus the Dependency Order section above it.
System Module Map	Superadmin, Buyer, Seller, Logistics, Rider, Mobile, Laravel API, Database	Logistics and Rider are mapped in full here; the other five modules (already built) are catalogued in the Loved-IT project state doc's codebase inventory — combine both for the complete map.
Team Responsibility Matrix	Backend, frontend, mobile, database, UI/UX, testing, documentation, deployment	The owner line under each module heading plus the per-task assignees give role coverage for Logistics/Rider; pair with the Team & Responsibilities table in the project state doc for the full-team view.

Built from the Development Workspace & Task Planning milestone brief and the Loved-IT project state doc (v1.1.0). Times are estimates for one person — split further if a task looks too big to verify as one unit, per the milestone's own planning rule. Copy each task's title, assignee, estimate, and details straight into the New task modal.